SECURITY · MIGRATION GUIDE

PHP Commerce: Moving Off Shared Hosting

Familiar is not the same as safe. What to move, where to, and in what order — after the October 2026 delistings and checkout hijack.

> **Familiar is not the same as safe.** Protecting customers now takes wholesale change, not patches — especially where personal data (PII), card data (PCI) or health data (PHI) is involved. The October 2026 checkout hijack ran through apps merchants had trusted for years. Shopify and Google have been raising these requirements since 2022. Most of the deadlines have passed; what remains is enforcement — delisting, blocked installs and lost ad eligibility.

Who this is for: merchants, agencies and app developers running WordPress and WooCommerce, Magento or Adobe Commerce, or PHP-backed Shopify apps — loyalty, upsell, product options, wishlists — on shared or single-VM hosting.

The question it answers: what to move, where to, and in what order, so that one breached plugin, one compromised vendor or one runtime flaw cannot reach every customer on the server.

The PHP runtime itself is covered in the companion article: patch for **php2shell** (CVE-2026-109618 and CVE-2026-109619) and keep the Cloudflare WAF rules in Block — Entitlement Security → php2shell. This article is about the architecture around it. For the owner and legal view of the same deadlines — what goes dark, and who carries the liability — see the Risk & Liability Brief: The 2026 Client-Side Cliff.

Vocabulary

TERM	MEANS	IS NOT
Shared VM	One server where many sites share an operating-system user, a database user and a filesystem	A private server with isolated apps
Private VM with Docker	One server the internet cannot connect to, with each application in its own sealed container, reached only through a Cloudflare Tunnel	Anything with an open inbound port
TLS transport	The sealed, tamper-evident envelope data travels in on every hop — shopper to edge, edge to server, server to Xano, server to database	Only the padlock in the browser
Sidecar	A small helper that runs beside each app and does one job — here, asking Xano `/auth/me` who the caller is and attaching a verified answer	The code-injection attack; it closes the hole where an attacker types someone else's customer ID
Real-time data binding	Prices, stock and points read live from the system that owns them	A copy refreshed by a nightly sync
AI task binding	An automated job limited to one tenant, one task and the tools that task needs	An agent holding a platform key
Client-side redirect	A script in the shopper's browser sending them to another domain	A 301 issued by the server

The requirements are not new

SINCE	REQUIREMENT	WHAT IT DEMANDS	REFERENCE
Oct 1, 2022	Shopify protected customer data (API 2022-10)	Personal data redacted by default; apps apply for access and meet Level 1 and Level 2 data-protection requirements	Shopify
Mar 2022, mandatory Mar 31, 2025	PCI DSS v4.0, requirements 6.4.3 and 11.6.1	An authorised inventory of every script on payment pages, integrity checks, and alerts on unauthorised change — the control that catches a checkout hijack	BDO summary
Mar 2024	Google Consent Mode v2 for EEA traffic	Consent signals for ad storage, analytics, ad user data and ad personalisation	Google
Aug 26, 2024	Shopify Customer Account API	At least Level 1 protected-customer-data approval	Shopify
Apr 1, 2025	New public Shopify apps GraphQL-only; REST Admin API legacy	New App Store submissions cannot use REST	Shopify Partners
Jun 30, 2026	Shopify Scripts stop executing	Checkout logic must be Shopify Functions	Shopify changelog
Oct 16, 2026, 15:00 UTC	Shopify API version 2025-10 no longer accessible	Apps still on unsupported resources after an upgrade deadline are delisted	Shopify versioning

October 2026: delistings and the checkout hijack

About **126 apps** were pulled from the Shopify App Store between October 14 and 18, 89 of them on October 15. Most of that number is versioning, not attack: API 2025-10 stopped being accessible on October 16, and delistings are routine — LetsMetrix counted 512 between September 22 and October 21, about 17 a day. Shopify has published no count and named no apps.

Only a few are tied to the hijack. Between October 10 and 20, attackers changed vendor-side settings so that shoppers clicking Checkout were sent to a lookalike payment page. **Shopify's own checkout was not breached.** The code came from vendors' configuration APIs and internal tools, not from the app files Shopify hosts — which is the risk of a third-party server holding store access.

APP	STATUS	BASIS
Hulk Product Options	Hijack confirmed (Oct 10)	Developer statement — cside
Selleasy	Hijack confirmed (Oct 13)	Developer statement — LetsMetrix
Swym Wishlist Plus	Hijack confirmed (about Oct 14)	Unauthorised configuration change — LetsMetrix
Rebuy Engine — PHP app	Delisted; **not** checkout hijacking, self-reported	Statement by James Van Erck, co-founder and COO, on LinkedIn, 22 October 2026: on Oct 14, attackers found an unprotected sign-up path that skipped inbox verification, registered accounts on the company's own email domain, and inherited staff privileges on internal development stores. They read product settings and data-source rules for 11 merchant stores; no edits, no API keys or tokens exposed; merchants notified within 24 hours; contained within hours. Not independently verified
Privy; Boost AI Search & Filter	Delisted Oct 18; link to the hijack unconfirmed	Tracker listings, no vendor confirmation
17TRACK; Consentmo GDPR Compliance	Delisted; link unconfirmed	Tracker listings; Consentmo's rests on a single confidential source

Rebuy's account is the identity half of the same lesson. Nothing in the runtime failed: an email address on the company's domain was treated as proof of being staff, and staff accounts carried standing access to stores. Its remediation — privileges only for a named group, multi-factor authentication for every merchant, a third-party penetration test of the authentication services —

is the permissions model this article argues for: identity verified at the server, never inferred from what the caller typed, and access granted per task rather than held by default.

The same mistake is not unique to apps. Identity providers and the software built on them have shipped it too:

CASE	WHAT WAS TRUSTED	WHAT IT ALLOWED	REFERENCE
Microsoft Entra ID — nOAuth (first disclosed June 2023; still present in about 1 in 10 of 100+ tested SaaS apps in June 2025)	An unverified email claim used as the user's identifier	With only an attacker-controlled Entra tenant and the victim's email address, full takeover of the victim's SaaS account. MFA, conditional access and Zero Trust policies did not stop it	Semperis
Okta AD/LDAP delegated authentication (advisory Nov 1, 2024; introduced July 2024)	A cached earlier login for usernames longer than 52 characters — often email addresses	Sign-in with the username alone, without the right password, where MFA was off	Okta advisory
Rebuy Engine (Oct 14, 2026, self-reported)	An email address on the company's own domain, never verified	Staff-level reads on internal stores and settings reads on 11 merchant stores	Co-founder and COO, LinkedIn, 22 Oct 2026

The rule all three break is the same: **an email address is a contact detail, not an identity.** Identify a user by the identity provider's immutable subject — the OpenID Connect `sub` with its issuer, in Entra the object ID with the tenant ID — require a verified email before linking any account, restrict which tenants may sign in, and grant elevated access only to a named group. MFA is necessary and, as nOAuth shows, not sufficient.

Why `/auth/me` claims and extras refuse the impostor

In the CRM Sync model, nothing an app does is decided from the email address. Every request is resolved through `/auth/me`, which reads five things in a

fixed order, and each one is a row someone had to be granted rather than a fact inferred from what the caller typed:

STEP	WHAT `/AUTH/ME` READS	WHAT IT PROVES	WHAT IT DOES NOT ACCEPT
1 · Token	A server-signed session token, checked against a logout denylist on every read	Which subject this is — the token's subject ID, nothing more	An ID, email or role sent in the request body
2 · Account status	The subject's row, before any write runs	The account is live, not deleted or tombstoned	A valid token for a deleted account
3 · **Claims** (`user_claims`)	Identity links keyed by the provider's immutable subject — Google's `sub`, the Shopify customer ID — and `email_verified` as the provider asserted it, one row per subject	Which external identities belong to this subject	An email match as proof that two logins are the same person
4 · **Extras** (`user_extras`)	Profile and preference fields, cleared when marketing consent is withdrawn	What personal data exists, and that it is consented	Profile data held by the app that renders the page
5 · Entitlement	Permissions granted to the subject, scoped to a shop	What this subject may do, and where	Privilege inferred from an email domain, a staff flag or an account's age

Platform administration sits outside this path entirely: a separate root credential and revocable tenant-scoped tokens, never a user account with an elevated flag.

INCIDENT	WHAT THE ATTACKER HAD	WHERE `/AUTH/ME` STOPS IT
Rebuy	A new, unverified account on the company's own email domain	Step 5. A new subject has no entitlement rows, and no rule anywhere grants permissions by email domain — so the account authenticates and can reach nothing. Step 3 records it as unverified, and no claim links it to any staff subject
Entra nOAuth	An attacker's tenant asserting the victim's email address	Step 3. Subjects are matched on the provider's immutable subject, not the email claim, so a different tenant's identity is a different subject with no claims, extras or entitlement of the victim's
Okta delegated authentication	A username and a cached earlier sign-in	Step 1. Identity is only ever a server-signed token for a subject ID, checked against revocation on every read; there is no username-plus-cache path to a session

Closed on 22 October 2026. Writing this section surfaced the same mistake in CRM Sync's own code, and it is fixed in production. An email-only match now attaches a login to an existing subject only when the inbox is proven — the provider verified the address, a Shopify one-time-code login, or an invitation link — and an unverified login is refused rather than attached. A typed email on a public form writes nothing until a single-use link mailed to that address is clicked, and every unconfirmed submission gets the same reply, so the form cannot reveal whether an address has an account. Rewards enrolment takes identity only from a session. Four static checks in the release gate fail if any of this regresses. The full claims-and-extras model is in Entitlement Security → claims and extras.

Merchant checks: find these apps among those installed, delisted ones included; review app embeds and custom JavaScript fields for scripts nobody added; run a test checkout in a private window and confirm it lands on Shopify's checkout domain; read Shopify's notice for your store's exposure window. Shoppers who paid by Shop Pay or PayPal were generally not exposed; shoppers who typed a card number during a window were.

Remove every client-side redirect to another domain

The hijack worked because a script in the shopper's browser could send them somewhere else. Any script that can do that today is a hijack waiting for a compromised vendor — legitimate ones included: market redirectors, affiliate rewriters, "faster checkout" buttons, leftovers from a previous storefront.

REMOVE — CLIENT SIDE	REPLACE WITH — SERVER SIDE
`window.location` or `location.href` in theme code, app embeds or tag managers	Redirect rules at the Cloudflare edge or a Worker: a 301 to an allow-listed destination
`<meta http-equiv="refresh">` pointing off-domain	The same edge redirect, or a link the shopper chooses
Scripts that rewrite links or the checkout button	Checkout posts only to the store's own Shopify checkout
Geo or market redirectors in the browser	Market routing at the edge, on Shopify Markets domains or subfolders
Old domains forwarded by script	Domain-level 301s in one redirect table under version control

Then hold it: a destination allow list for every server redirect; a Content Security Policy with `script-src` limited to an inventoried list and `form-action` limited to the store's own origins and checkout; a payment-page script inventory with change alerts (PCI DSS 6.4.3, 11.6.1); and a private-window test checkout on every release.

What is being replaced

RISK ON A MAMP/WAMP-STYLE SHARED VM	WHAT HAPPENS	WHAT REPLACES IT
One OS user for every site	A file-read or upload bug in one plugin reads every site's `wp-config.php` or `env.php`	One container per site, read-only filesystem, no shared volumes
Public origin IP	Attackers skip the CDN and WAF	Origin reachable only through a Cloudflare Tunnel
TLS ends on the box, or not at all	Plain HTTP between CDN and origin	Edge certificate, Full (strict), origin certificate or tunnel
One database user	Every app can read every table	A role per service, plus Postgres row-level security
Admin panels on the open web	`/wp-admin`, Magento admin and phpMyAdmin face everyone	Cloudflare Access on every admin path
Frozen PHP versions	End-of-life PHP survives because upgrading breaks a neighbour	A pinned image per site, rebuilt and scanned each release
Secrets in files	Keys inside the web root	Secrets injected at runtime
No audit trail	FTP edits leave no record	Git commit, image digest and deploy tag per change

The options, and what each one isolates

OPTION	TLS AND ORIGIN	CODE ISOLATION	ADMIN LOGIN	DATA ISOLATION	SCALING
Shared VM (today)	TLS on the box or none; public IP	None	Passwords on the open web	One database user	Up only
WordPress or Magento in Docker behind Cloudflare	Edge TLS, Full (strict); tunnel; no inbound ports	Container per app	Cloudflare Access	Role per service	One VM, up
EmDash on Cloudflare Workers	Edge TLS; no origin	Isolate per plugin, permissions manifest	Passkeys; SSO	Plugins have no direct database access	Pay per use
11ty + Decap CMS on Cloudflare	Edge TLS; no origin	No code runs per request	Access on `/admin` + GitHub OAuth	Static files; live data through APIs	Static requests free and unlimited
Jekyll on GitHub Pages	GitHub-managed TLS	No code runs per request	GitHub accounts	Static files	Free within 1 GB and a 100 GB/month soft limit; **not permitted for e-commerce**
Shopify Storefront Web Components on Webflow, WordPress, Astro/EmDash, 11ty or AEM	The site's own TLS edge; checkout on Shopify	No commerce code on the site — Shopify's script renders products and cart	The site's own; customer accounts on Shopify	The site stores no customers, orders or cards	Scales with the site; commerce scales on Shopify

OPTION	TLS AND ORIGIN	CODE ISOLATION	ADMIN LOGIN	DATA ISOLATION	SCALING
Shopify Functions in Rust	Runs inside Shopify	WebAssembly sandbox, declared input only	Shopify app permissions	Scoped to the shop	Shopify scales it
Kubernetes + Postgres RLS	Edge TLS; NetworkPolicy; TLS to the database	Pod and namespace	Access + SSO	Row-level security per tenant and actor	Horizontal; highest cost

Order of moves: static first, for every page that needs no login. EmDash next, where editors need a live CMS. WordPress or Magento in Docker last, only for what cannot yet leave PHP.

EmDash was announced by Cloudflare on April 1, 2026 as a v0.1.0 developer preview — MIT licensed, TypeScript on Astro, no WordPress code. Content imports from a WXR export or the EmDash Exporter plugin; themes must be ported. It has no commerce, so WooCommerce stays in Docker or the store moves to Shopify. Pilot it on a blog subdomain first. Cloudflare: EmDash

The static path has run in production for years — Jekyll and GitHub Pages since 2008, 11ty since 2017, Decap since 2016 as Netlify CMS. GitHub Pages' terms forbid running an online business or e-commerce site on it, so a store built with Jekyll or 11ty is hosted on Cloudflare.

The safer customer option: let Shopify hold the customer and the card

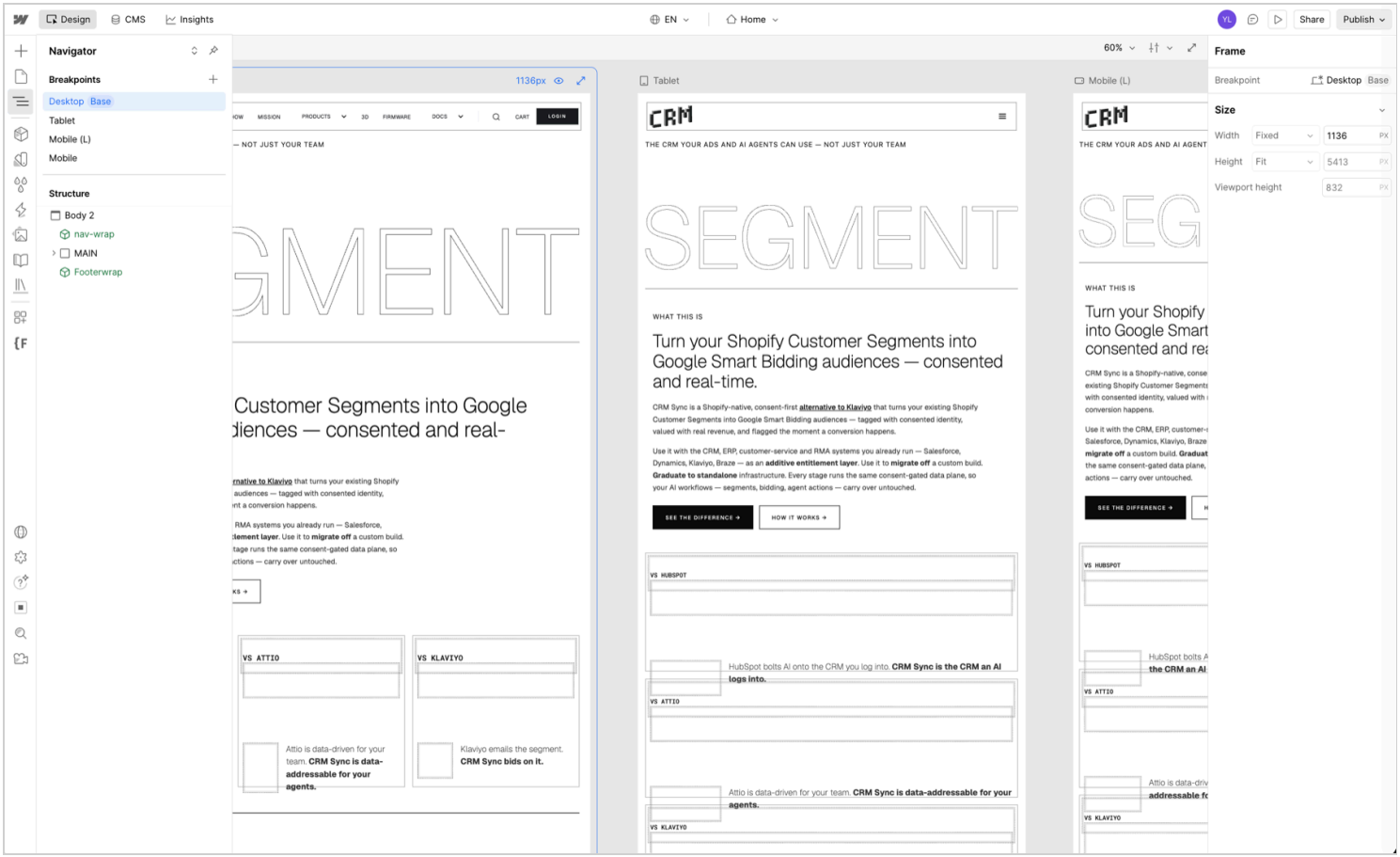
For a store that wants to keep its WordPress or Webflow site, the most effective PII and PCI reduction is to stop holding the data at all. Shopify's Storefront Web Components put products, prices and a cart on any page with a few lines of HTML, and the **Buy now** button sends the shopper to checkout on Shopify.

Shopify's own WordPress plugin — built by Shopify, not by WordPress — does the same from the WordPress Editor: products are managed in Shopify and placed on WordPress pages and posts.

The components are plain HTML elements plus one script, so they are not tied to a CMS. The same block works in a Webflow embed, a WordPress theme or block, an Astro or EmDash component, an 11ty template, and an AEM component or Edge Delivery block — anything that renders HTML and can load a script. Try them in Shopify's playground.

A working example. crm-base.webflow.io is a Webflow site with no commerce code of its own. When a shopper adds a product to the cart, a CRM Sync embed mounts Shopify's `<shopify-store>` and `<shopify-cart>`, and checkout lands on the Shopify store at www.crm-sync.dev. The Webflow site never sees a card number or keeps a customer record; the Shopify store holds both.

The same theme renders the same everywhere. The design is the Webflow export plus one embed block, with no framework code in it, so the same theme renders the same on WordPress, AEM, 11ty, Shopify, Next, Astro, Nuxt, Svelte and Drupal — with the forms and the data-binding security intact. The security does not live in the theme, so it does not change when the host does: forms still post to the Worker and hold a typed email until the inbox confirms, `/auth/me` still decides who the caller is, permissions are still granted rows checked on every request, and the cart and checkout are still Shopify's. None of those hosts ever holds a customer record, a consent record or a card. The WordPress and Astro/EmDash paths are in File System Agnostic Publishing, and the AEM path in Shopify / Google Integration on an AEM Scaffold.



The crm-base design in Webflow at three breakpoints. The structure is a navigation component, the page content and a footer component — design only. The CART link in the navigation opens Shopify's cart; the products, cart and checkout come from Shopify, not from Webflow.

OPTION	WHAT THE SITE HOLDS	WHAT SHOPIFY HOLDS	WHY IT IS SAFER
Storefront Web Components on Webflow, WordPress, Astro/EmDash, 11ty or AEM	Markup, the store domain, and optionally a public, read-only Storefront token (needed only for inventory counts and custom data)	Catalog, prices, cart, checkout, customer accounts, payments	No customer table, no order table, no card flow and no secret key on the site; a breach of the site has no customer database to take
Shopify's WordPress plugin	WordPress pages and posts that embed Shopify products	Products and checkout; Shopify states PCI, SOC and GDPR compliance are built in	WooCommerce's customer and order tables never exist in the WordPress database
WooCommerce or Magento on the site's own server (for contrast)	Customers, orders, addresses, often payment-flow scripts	Nothing	Every row of PII sits in the same PHP application as every plugin

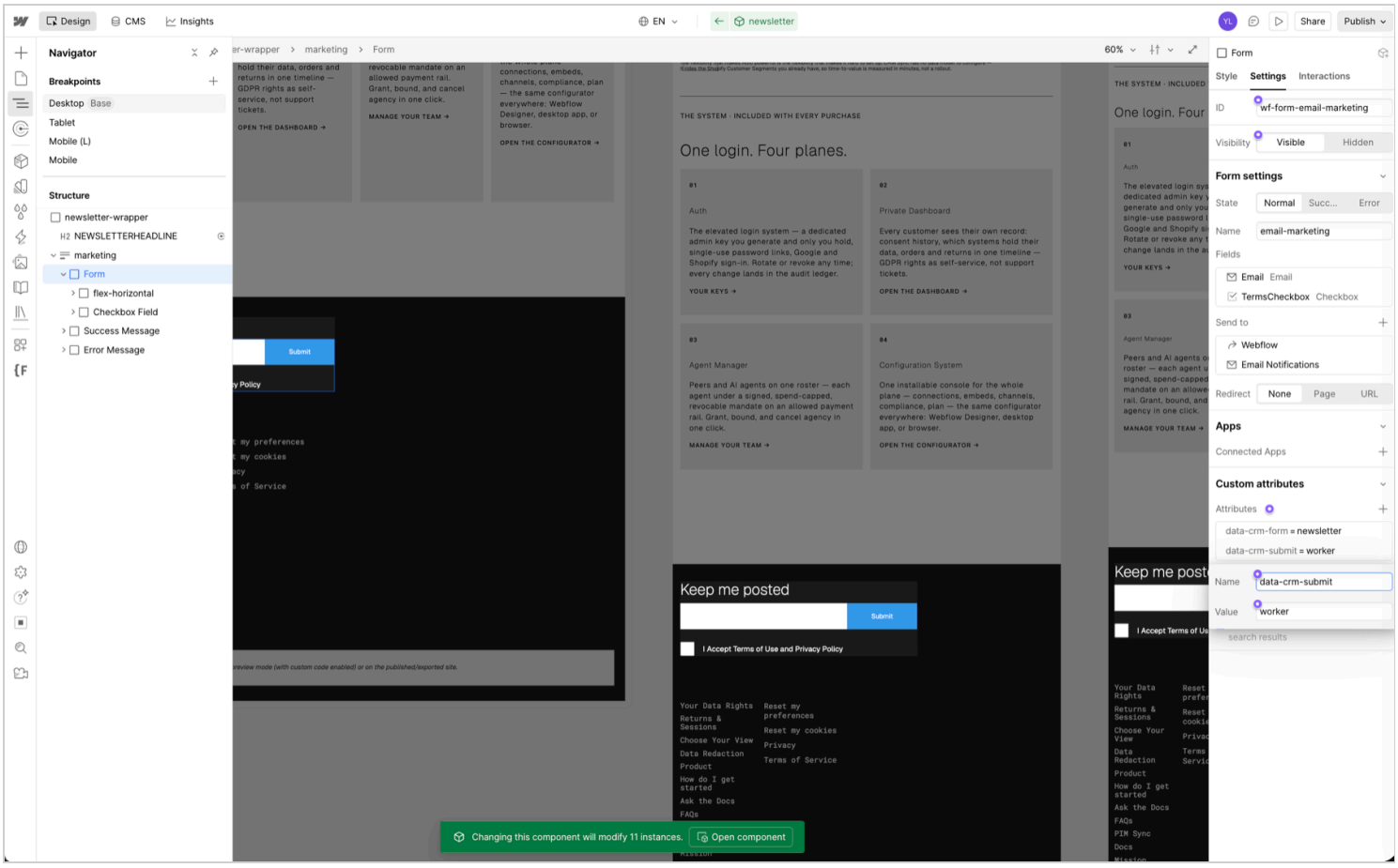
What it does not remove. The page that holds the Buy now button is still a page you serve, and the October 2026 hijack worked by changing what happened *before* checkout. The rules above still apply to it: no client-side redirects to other domains, a Content Security Policy that lists every script, an inventory of the scripts on any page that links to checkout, and a private-window test checkout on every release. WordPress still runs PHP for the content site, so the runtime patches still matter. Only the public Storefront token belongs in the page — never an Admin API token. Shopify's plugin page does not say where customer and order records live; confirm that before describing the plugin to customers as keeping them off WordPress.

Setup, including the Webflow-to-WordPress and EmDash/Astro paths, is in File System Agnostic Publishing → the Web Components block.

Secure forms on any front end

A form is the other place PII enters, and the platform's own form storage — Webflow submissions, a WordPress form plugin's table, a Liquid contact form — puts that PII next to everything else the platform runs. CRM Sync forms send the data to the Worker instead, and the form itself is only HTML:

```
<form data-crm-form="newsletter" data-crm-submit="worker">
  <input type="email" name="email" required>
  <label><input type="checkbox" required> I accept the Terms of Use and Privacy Policy</label>
  <button type="submit">Submit</button>
</form>
<script src="https://crm-sync.dev/embed/forms.js" defer></script>
```



The crm-base newsletter form in Webflow. The design stays in the Designer; the two custom attributes hand the submission to the Worker.

Because the contract is two attributes and one script, the same form works wherever HTML renders:

FRONT END	WHERE THE FORM LIVES
Webflow	A form element with the two custom attributes, as above
Astro / EmDash	A component that renders the `<form>`; the script in the layout
Next.js	A component that renders the `<form>`; the script through `next/script`
11ty	An include or shortcode; the script in the base layout
Shopify	A theme section or snippet in Liquid; the script in `theme.liquid`
WooCommerce / WordPress	A block or template part; the script enqueued by the theme
AEM	A component or Edge Delivery block that emits the `<form>`

What makes it secure.

- **The platform never stores it.** The script intercepts the submit before the platform's own handler runs and posts to the Worker, so the address is not written to Webflow, WordPress or Shopify form storage.
- **A typed email proves nothing.** Without a session, the Worker records nothing and mails a single-use, 48-hour confirmation link; the permission is written — with its date, time and the exact wording shown — only when the inbox answers. A signed-in shopper's session proves the address, so theirs is recorded immediately.
- **The reply never reveals an account.** Every unconfirmed submission gets the same answer, whether or not the address already has an account.
- **Abuse is capped.** 60 submissions an hour per IP, three confirmation mails a day per address.

The demo, on our own data. This is the live run from 22 October 2026, submitted from no session with the author's own address (masked here):

```
POST https://crm-sync.dev/forms/newsletter

{"email":"y••••••••••@gmail.com","page":"/verify-email-is-not-identity","locale":"en"}
```

```
{"ok":true,"form":"newsletter","form_id":"newsletter_en_2026-10-22T12:02:22.862Z_f767c744",

 "locale":"en","status":"confirmation_sent",

 "why":"check your inbox — the request is recorded once the address confirms it"}
```

No user ID, no "created" flag, and no consent row until the link in the inbox is clicked. The `form_id` is the join key: the same value is written to the reconcile log, the GA4 signal event and the response, and it survives confirmation, so the held entry and the recorded one join on it.

Email-only plugin forms and fee-based middleware. Most CRM and plugin forms capture an email address and nothing else — no consent wording, no time, no session — and store it in the vendor's own database, often a PHP application outside the merchant's control. Fee-based middleware then copies it onward, charging per task or per record. Moving that audience to Google is not a file transfer: Google's EEA consent rules require `ad_user_data` and `ad_personalization` consent for the people in it, and an address with no consent event behind it has nothing to show. Migrate with the consent, not just the address — export what consent evidence exists, send the same single-use confirmation to everyone whose record has none, and connect the confirmed audience to Google's APIs directly from the Worker, which also ends the per-record middleware fee.

What it does not cover. If the script is blocked — an ad blocker, a strict CSP that omits it — the platform's own handler takes the submit, and the address lands in the platform's form storage after all. Remove the platform's own destination

where the platform allows it, list `crm-sync.dev` in the CSP, and test the form with the script blocked.

Packaging the form. The HTML contract above is what ships today. Because the form is a contract — a slug, its fields, its consent wording — it can travel in other shapes:

SHAPE	WHAT IT CARRIES	STATUS
HTML + `forms.js`	The form and the submit interception	Built
Worker route `/forms/<type>`	Validation, the hold, the confirmation mail, the consent record	Built
`form_id` join key	One identifier across the reconcile log, GA4 event and consent record	Built
A Rust crate	The form's field schema and validation as a type, compiled to WebAssembly and checked inside the Worker — the same shape check used for AI task binding	Pattern, not built
Joinable data for 11ty, Liquid or Markdown	The form definition as data — `_data/forms.json` in 11ty, a Markdown front-matter block, or a Liquid snippet that reads it — so every front end renders the same fields and wording from one record	Pattern, not built
A Shopify metaobject	The same definition as a metaobject entry, so the theme renders the form from data the store already governs	Pattern, not built

Shopify plugins become Rust Shopify Functions

OLD PLUGIN PATTERN	REPLACE WITH	WHY
Line-item Script — tiers, BOGO, loyalty redemption	Discount Function	Runs inside Shopify on declared input; no merchant server to breach
Shipping Script	Delivery customization Function	Same
Payment Script	Payment customization Function	Same
Upsell or bundle app editing the cart	Cart transform Function + theme app extension	No third-party server holding order scopes
checkout.liquid fields and validation	Cart and checkout validation Function + Checkout UI extension	No arbitrary script on the payment page
Thank-you page script tag for points or pixels	HMAC-verified order webhook to a ledger Worker, plus a consent-aware web pixel	Events verified; tracking honours consent
PHP app server on the REST Admin API	GraphQL Admin API from a Worker	Meets the GraphQL-only rule; no PHP host

Shopify recommends Rust for public apps, large carts and complex logic, because it compiles directly to WebAssembly and stays inside the instruction-count limits; JavaScript is for prototypes. Functions decide; they do not store. Points, history and consent stay in the ledger, behind `/auth/me`.

Identity, data and AI tasks

1. **Identity.** A sidecar resolves every request through Xano `/auth/me` with the caller's JWE token and forwards the actor in a signed internal header. PHP never parses tokens and never trusts a `user_id` sent in a form.

2. **Data.** Postgres row-level security filters every read and write by tenant and actor, so a plugin bug or SQL injection returns only the caller's own rows. MySQL has no RLS; WooCommerce and Magento data either moves or sits behind filtered views.
3. **AI tasks.** Each automated job runs in a Tool Runner loop with a scoped tenant key and only the tools that task needs — the runner exists in PHP too. The full model — Cloudflare binds, Xano grants, Rust checks — is in Entitlement Security → AI task binding.

AI data binding with provisional permissions. A task binding says which tools an agent may call. The data it may touch is bound the same way — by permissions that expire, and by gates on where the subject is, who they share with and what device they used:

GATE	WHAT IT CHECKS	STATUS
Provisional access	Every grant carries a start and end date, and each permission can carry its own expiry; both are checked on every permission decision, so a time-limited grant ends without anyone revoking it	Built
Sovereign nation	A per-country policy, keyed by ISO code, read before an order exists: sovereign markets require a named permission (Korea's domestic settlement, for example), sanctioned or blocked markets are refused, and an unreachable policy fails closed	Built
Peer	Whether one subject may share with another	Partial — the check is written and tested but no route calls it yet; sharing is governed by single-use, expiring invitations today
Device	The platform, device type and browser behind each consent event	Recorded as provenance on every consent row; refusing access by device is not built

HOP	PERMISSIONS	TLS TRANSPORT
Browser or agent → edge	Access policy, or the Xano JWE	TLS 1.2 minimum; HSTS
Edge → origin	Origin accepts only Cloudflare	Tunnel; no inbound ports
Runner → tools	Scoped tenant key, per-task tool list	HTTPS; Access service tokens
Worker → Xano	Workspace-scoped key; permission check in Xano	HTTPS; a shared-secret header so the direct Xano URL refuses traffic that skipped the Worker
Shopify → store logic	Function input query; webhook HMAC, constant-time compare	Shopify-hosted; HTTPS webhooks
Service → database	Role per service; actor and tenant set per request	`sslmode=verify-full`

Google Merchant API: rebuild loyalty, reviews and comparison in one pass

The Google feed integration is being rebuilt regardless. The Content API for Shopping shut down on 18 August 2026; Merchant API became generally available as v1 in July 2025, its v1beta versions were discontinued on February 28, 2026, new features ship only there, and conversion sources move from `content/v2.1` to `conversions/v1`. Connect loyalty, reviews and price comparison directly in the same conversion, rather than through a plugin that can be delisted.

GOOGLE SURFACE	MERCHANT API	NEEDS	REFERENCE
Loyalty — member prices, points, loyalty shipping	`loyaltyPrograms[]` on product attributes	A program linked in Merchant Center; AU, JP, UK, DE, FR, US; member prices matching feed, landing page and checkout	Product attributes · Attribute spec
Product reviews	Reviews sub-API, `productReviews`	Product ratings enrolment; `is_verified_purchase` set from a real order	Guide
Seller reviews	Reviews sub-API, `merchantReviews`	Store ratings enrolment	Guide
Price comparison	Reports sub-API — price competitiveness, price insights, competitive visibility	Standalone or individual sub-account	Reports
Comparison Shopping Services	CSS API; `Csses` folds into Accounts	A CSS partner	CSS
Conversions	Conversions sub-API, `conversionSources`	Field renames: `property`, `destination`, `report`	Migrate

Conversions at no ad spend. Merchant Center conversion sources report conversions from free listings — the organic surfaces where reviews, loyalty member prices and price comparisons appear — as well as from ads. They only match exactly when each event carries three things, captured at the moment it happens:

CAPTURED	WHY IT MAKES THE MATCH EXACT
The GA4 session ID	Ties the conversion to the visit that produced it, not to a guess about the visitor
Device provenance	Platform, device type and browser, recorded with the consent event, so the same person on two devices stays two provable sessions
An idempotency key	Derived from the subject or session, the event, its time and its source (`form_id`), so a retry or a double-fired tag is one conversion, never two

That is what turns an AI answer into an attributable sale: the assistant finds the candidate, and the session, device and key decide the transaction. CRM Sync writes all three today for a signed-in submission. A submission confirmed by email deliberately drops the typing browser's GA identifiers — carrying them would bind a stranger's browser to the person who clicked the link — so it keeps the device and the key but not the session. Conversions sent without all three are estimates.

A verified-purchase review or a member price shown to Google is a claim about a real order and a real member. CRM Sync records that claim as a signed mandate anyone can check offline in a browser — verify a mandate · verified transactions.

Supply chain: every new ecosystem is a new door

Moving off PHP brings in Python for AI gateways, npm for Workers and Rust crates for Shopify Functions. On March 24, 2026, LiteLLM 1.82.7 and 1.82.8 were published to PyPI with a credential stealer — one through a `.pth` file that runs on every Python start — taking SSH keys, cloud and Kubernetes credentials, `.env` files and TLS private keys (BleepingComputer). A second poisoned release, 1.107.0, followed on October 21 — Entitlement Security → LiteLLM.

ECOSYSTEM	PERMISSIONS CHECK BEFORE INSTALL	WHAT IT CATCHES
Rust crates	`cargo vet --locked` in CI: every version covered by a trusted published audit, or reviewed and certified	A hijacked release nobody has reported yet
Python	Hash-pinned installs; versions at least two weeks old	A new malicious release
npm	Committed lockfile; `npm ci --ignore-scripts`; an osv-scanner baseline with expiry dates	Install-time scripts; known-bad versions
GitHub Actions	Pinned to commit SHAs	A retagged action

CRM Sync's desktop build imports the published crate audits of Mozilla, Google, the Bytecode Alliance, ISRG, Embark and Zcash — 102 crates fully audited, 357 recorded as a starting baseline — and fails CI on any crate version no audit covers. Wildcard trust is avoided on purpose: a hijacked publisher would pass it.

No silver bullet: rotate, randomise, look forward

No single control in this article would have stopped every incident in it. TLS did not stop the checkout hijack. MFA did not stop nOAuth. A WAF does not stop a poisoned package, and a package audit does not stop an unverified sign-up. What holds is layers that change faster than an attacker can map them — so the defence has to move on a schedule, and be tested in ways the attacker cannot predict.

PRACTICE	WHAT IT TAKES AWAY FROM THE ATTACKER	CADENCE
Rotate every credential — on a schedule, and immediately on any incident, through a recorded key ceremony	The useful life of a stolen key	Every 90 days, and on event
Short-lived sessions, revocation checked on every read	Replaying a captured session	Every request
Keys generated per deployment, never copied between environments	One leak opening every environment	Every deployment
Unguessable identifiers — random, not sequential — in anything a public response returns	Enumerating accounts, orders and records by counting	Always
A fresh Content Security Policy nonce per response	Reusing an injected script	Every response
Testing the attacker cannot predict: rotating penetration-test scope, AI-assisted scanning for chained attacks, canary credentials that alert the moment anyone uses them	Mapping a defence that never changes	Continuous
A delay and an audit gate on every dependency	The first hours of a malicious release	Every update

Build to the standard that is coming, not the one being enforced today. Every requirement in the table at the top of this article was announced years before it was enforced, and the merchants hurt worst in October 2026 were the ones still meeting the old version. The ones already on the calendar: the Content API for Shopping shut down on 18 August 2026; EU AI Act transparency obligations apply from 2 August 2026; the EU Cyber Resilience Act's reporting obligations began on 11 September 2026, with full obligations from 11 December 2027; GS1 Sunrise 2027 moves retail point of sale to 2D barcodes. Each date, with its source, is in The Compliance Calendar.

What it costs

COST	WHAT IT MEANS	SEVERITY
Rework of vendor plugins	Loyalty, upsell and options apps are rebuilt as Functions, Workers and a ledger, not configured	High — weeks per plugin, not hours
A monthly bill above shared hosting	A private VM, Cloudflare plans or managed Kubernetes cost more than a shared box	Low to high by tier; get current quotes
EmDash maturity	A v0.1.0 preview with a near-empty plugin ecosystem; WordPress's co-founder argues the isolation only holds on Cloudflare	Medium — pilot only
MySQL has no row-level security	WooCommerce and Magento data needs a move to Postgres or filtered views	Medium
New dependency ecosystems	Rust, Python and npm each need their own gate	Medium — the gates are the cost
Familiar tooling retired	Teams lose cPanel, FTP and plugin-store installs	Real, and the point

> **What this does not claim.** It is not a certification and not legal advice. Delisting counts come from public trackers, not Shopify. "Unconfirmed" means exactly that. Prices are deliberately absent: they vary by provider, region and traffic.

What to do next

Ordered by exposure — card data first.

1. **Payment pages.** Inventory every script, remove every client-side redirect, set the CSP, run a private-window test checkout. *Owner: store owner with the developer.*

2. **Delisted and hijack-listed apps.** Remove or replace; revoke their Shopify scopes after cutover. *Owner: store owner.*
3. **PHP runtime.** Patch for php2shell and keep the WAF rules in Block. *Owner: hosting or DevOps.*
4. **Origin.** Cloudflare in front, Full (strict), Access on every admin path, tunnel, inbound ports closed. *Owner: DevOps.*
5. **Containers.** One pinned image per app, secrets injected at runtime. *Owner: developer.*
6. **Identity and data.** `/auth/me` sidecar; a database role per service; RLS or filtered views; personal fields encrypted with a key generated for this deployment. *Owner: developer with compliance.*
7. **Checkout logic.** Scripts and checkout.liquid patterns rebuilt as Rust Shopify Functions. *Owner: Shopify developer.*
8. **Google.** Merchant API migration with loyalty, reviews, comparison and conversions in the same pass. *Owner: marketing operations with the developer.*
9. **Content.** Static publish first, EmDash pilot second. *Owner: content lead.*
0. **Decommission** the shared VM after the tunnel has carried production traffic for a full billing cycle. *Owner: DevOps.*

Evidence basis: public vendor, platform and standards documentation linked inline, read on 22 October 2026. Not a certification.
